BOSS · GAS COMPARISON

GAS와 ③ 패턴 구조 — 무엇이 같고 무엇이 없나

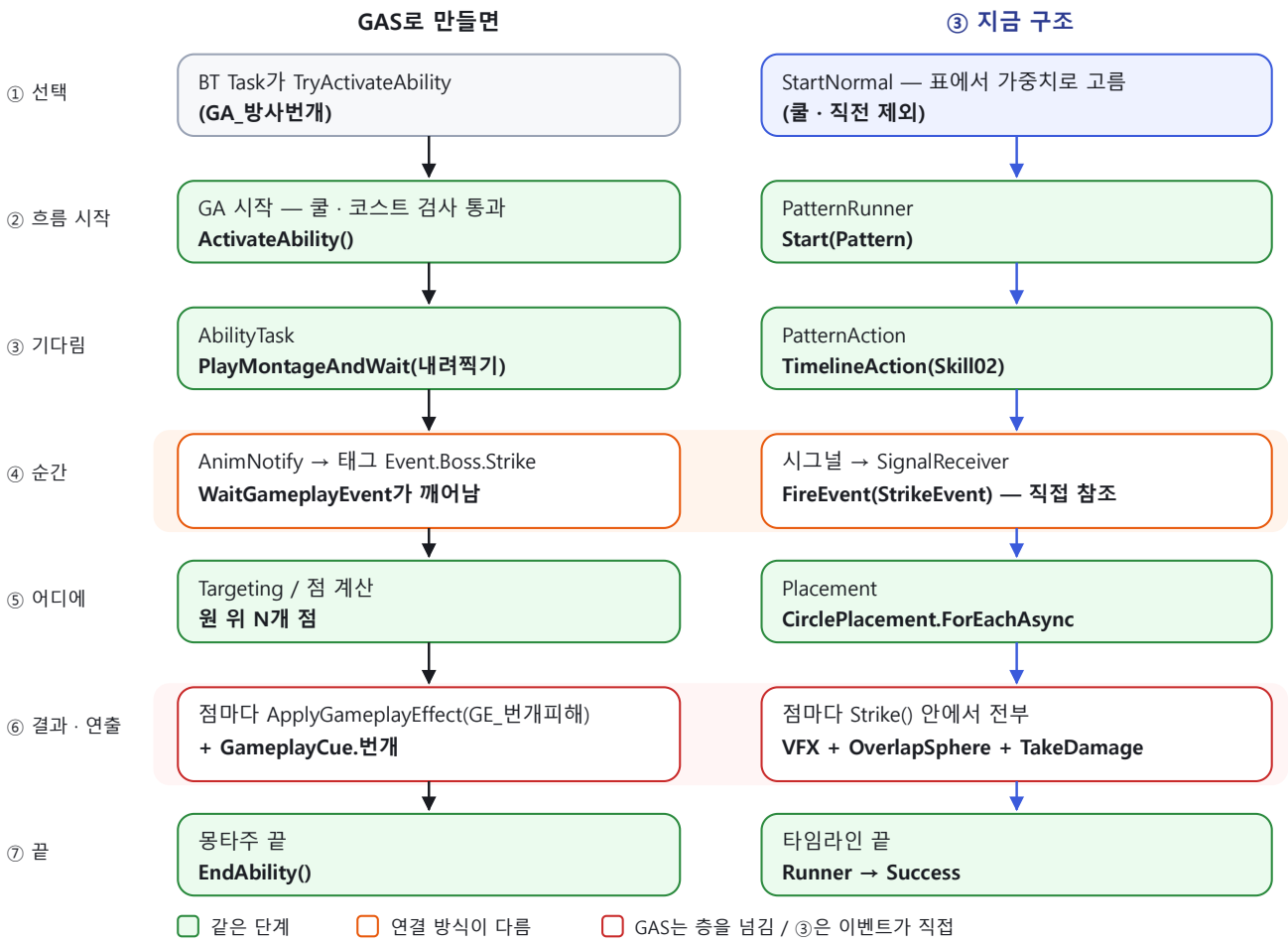
언리얼 **Gameplay Ability System**의 층을 지금 보스 구조(Pattern · Action · Event · Placement)에 하나씩 대어 봄

	GAS (언리얼)	판정	③ 지금 보스 구조
선택 누가 · 언제	**AI (BT · StateTree)** GAS는 "쓸 수 있나"만 판정	GAS 밖	**BT 5노드 + 일반 · 기믹 표** 가중치 · 쿨 · 직전 제외 · HP 줄
흐름 순서 · 끝 · 취소	**GameplayAbility** Activate → End · Cancel	같음	**Pattern + PatternRunner** Start → Tick → Success · Cancel
기다리는 동작 끝날 때까지	**AbilityTask** PlayMontageAndWait · WaitDelay	같음	**PatternAction** AnimationAction · TimelineAction
순간 애니 속 한 프레임	**AnimNotify → GameplayEvent** 태그로 보냄 · Ability가 기다림	비슷	**타임라인 시그널 → FireEvent** 이벤트 컴포넌트를 직접 가리킴
어디에 위치 · 대상	**Targeting System** AOE 모양 → 필터 → 정렬	비슷	**Placement** 모양이 점을 "언제 · 어디에" 내놓음
결과 맞으면 어떻게	**GameplayEffect (데이터)** 즉시 · 지속 · 수치 계산 한 곳	없음	**— 층 없음** StrikeEvent 안에 피해 코드 복사
연출 VFX · SFX	**GameplayCue** 태그로 VFX · SFX 재생	없음	**— 층 없음** Event가 직접 Instantiate · Play
주인 누가 들고 있나	**AbilitySystemComponent** Owner · Avatar · AttributeSet	비슷	**PatternContext** 칸 7개 만능 구조체 + GetComponent

한눈에 보면

1. **뼈대는 같다 — 흐름 · 기다리는 동작**
   - Ability ↔ Pattern, AbilityTask ↔ PatternAction — "흐름은 코드, 기다림은 Task"라는 결론이 GAS와 같음
2. **반쯤 같다 — 순간 · 어디에 · 주인**
   - 역할은 나눠 놓았지만 연결 방식(직접 참조)과 컨텍스트(만능 구조체)가 GAS보다 거침
3. **아예 없다 — 결과 · 연출**
   - GAS가 데이터로 빼 둔 두 층을 지금은 Event가 혼자 다 함
4. **선택은 GAS에도 없다**
   - GAS는 "쓸 수 있나(쿨 · 코스트 · 태그)"만 판정하고, 무엇을 쓸지는 AI가 고름 — ③의 표 · BT가 맡은 자리

1 같은 패턴 따라가기 — 방사형 번개



1. 일곱 단계 중 다섯은 1:1로 겹침

- 선택 → 흐름 시작 → 기다림 → 어디에 → 끝 — 순서도, 각 단계를 맡는 부품의 크기도 같음

2. ④ 순간 — GAS는 "이름(태그)"을 보내고, ③은 "대상"을 가리킴

- GAS 몽타주는 누가 받는지 모름 → 같은 몽타주를 다른 Ability가 다른 일로 재사용 가능
- ③ 타임라인은 StrikeEvent 컴포넌트에 묶임 → 같은 타임라인에 다른 이벤트를 달 수 없음

3. ⑥ 결과 · 연출 — GAS는 넘기고, ③은 혼자 다 함

- 피해 계산 · 상태 부여 · VFX가 StrikeEvent 안에 있음 → 피해 주는 이벤트가 늘면 그만큼 복사

2 GAS에서 가져오지 않을 것

1. 네트워크 복제 · 예측

- GAS 복잡도의 큰 몫 — 이 게임은 완전 싱글(네트워크 코드 0)이라 필요 없음

2. 계층형 태그 시스템 전체

- "Event.Boss.Strike" 같은 계층 · 쿼리까지는 과함 — 이벤트를 부를 이름(키) 하나면 충분

3. 쿨다운을 GameplayEffect + 태그로

- GAS는 쿨도 "태그를 붙이는 효과"로 처리 — 보스는 표의 쿨 Dictionary로 충분하고 더 읽기 쉬움

4. AttributeSet · Modifier 집계

- MonsterStat이 이미 있음 — 스탯 구조를 새로 만들 이유가 없음

GAS를 통째로 흉내 내지 않고, "층을 나누는 방식"만 가져온다

3 GAS에서 가져올 것 — 여섯 가지

선택	BT + 일반 · 기믹 표	F. 디버그 표시 · 다음 패턴 강제
흐름	Pattern + Runner + Action	D. 패턴 단위 취소 토큰
순간	타임라인 마커 → 키 → 지금 패턴의 Event	B. 이벤트 키 (태그 대신)
어디에	Placement — 지금 그대로 (점에 방향 · 번호 추가)	
결과	HitEffect (데이터) — 피해 · 상태를 한 곳에서	A. 결과 층 신설
연출	CuePlayer — VFX 생성 · 재생을 한 곳에서 (선택)	E. 연출 분리 (선택)

주인 = Owner (ASC 자리) — Animator · Director · Stat · Get<T>() 를 들고 모든 층에 공급 ← C

흐름 · 기다림은 이미 GAS 모양 → 고칠 곳은 "연결"과 "빠진 두 층"

A 결과 층 신설 — HitEffect (GAS의 GameplayEffect)

1. "맞으면 어떻게 되나"를 데이터 하나로

- 피해 배율 · 상태 부여 목록을 SO에 — 계산과 적용은 HitEffect.Apply 한 곳

2. Event는 "누구를 맞혔다"까지만

- StrikeEvent의 _damageMul · DamageInfo 생성 코드를 지우고 _onHit 칸 하나로

```
public class HitEffect : ScriptableObject          // GAS: GameplayEffect
{
    [SerializeField] float _damageMul = 1f;
    [SerializeField] List<…> _status;              // 상태 부여 (② 결과 Effect 재사용 자리)
    public void Apply(IPatternOwner src, IDamageable dst, Vector3 at) { … }  // 계산은 여기 한 곳
}
// StrikeEvent.Strike:  foreach (hit) _onHit.Apply(ctx.owner, hit, pos);
```

3. 왜 먼저 해야 하나

- 남은 시그널 5개(평타 · Skill1~3)를 ③으로 옮기면 피해 주는 이벤트가 늘어남 — 결과 층이 없으면 피해 코드가 그만큼 복사됨
- ②의 결과 Effect(피해 · 상태)는 원래 잘 맞던 부분 → 여기서 살려 씀

= 미뤄 둔 "어떻게" 층(데미지 파이프라인)의 첫 걸음

B 순간 연결 — 이벤트 키 (GAS의 AnimNotify → GameplayEvent)

지금



바꾼 뒤



1. 타임라인은 "지금 Strike 순간"이라는 이름만 보냄

- 누가 받는지는 실행 중인 Pattern이 정함 — GAS의 WaitGameplayEvent와 같은 자리

2. 같은 타임라인을 다른 이벤트로 재사용

- 텍스처 반전 보스(⑥)가 같은 Skill02 타임라인에 다른 Placement · 다른 HitEffect를 다는 게 표 수정만으로 가능

= 다음 할 일 ⑤ 마커 + ④ 시그널 5개 이관을 같이

C 주인 — Owner (GAS의 AbilitySystemComponent)

```
// 지금: animator · director · spawnPoints · target · self · token · summonedRegistry  (칸 7개)
public struct PatternContext { IPatternOwner owner; Transform target; CancellationToken token; }
public interface IPatternOwner                    // GAS: AbilitySystemComponent
{
    Transform Self; Animator Anim; PlayableDirector Director; MonsterStat Stat;
    T Get<T>();                                   // 보스 전용: ISpawnPointProvider · ISummonRegistry
}
```

1. 컨텍스트는 보편 정체성 3칸만

- 보스 전용 칸(spawnPoints · summonedRegistry)은 Owner.Get<T>로 — ①의 "스킬마다 칸이 늘어남"이 돌아오는 걸 막음

2. HitEffect가 스탯을 Owner에서 받음

- StrikeEvent의 GetComponent<MonsterStat>() 임시 코드 제거

= 다음 할 일 ① Context 최소화

D · E · F 나머지 셋

1. D. 패턴 단위 취소 — EndAbility가 Task를 정리하듯

- Runner.Start에서 전투 토큰에 연결된 패턴 토큰 생성 → 그 패턴이 쏜 Event가 이 토큰을 씀
- 무력화 · 사망으로 패턴이 끊기면 아직 내리치던 번개까지 같이 멈춤

2. E. 연출 분리 — GameplayCue (선택)

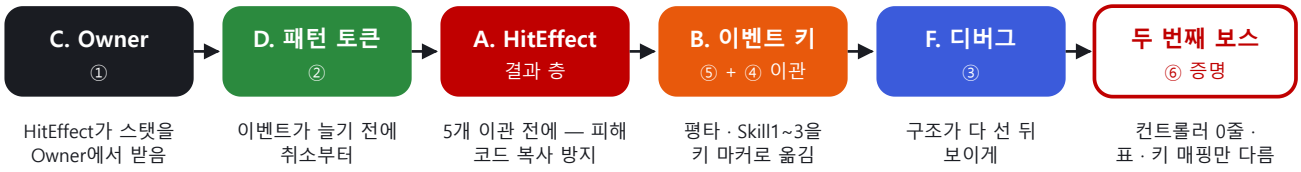
- CuePlayer.Play(프리팹, 위치) 한 곳에서 풀 생성 · Clear · Play — 이벤트마다 같은 VFX 코드를 반복하지 않게
- 이벤트가 2~3개뿐인 지금은 급하지 않음

3. F. 디버그 — showdebug abilitysystem처럼

- 화면에 지금 패턴 · 스텝 · 쿨 남은 시간 · 다음 기믹 줄 표시
- "다음 패턴 강제" 토글 — 할로우 나이트 모드 도구(FsmMaster)의 "랜덤 선택을 정해진 순서로"와 같은 발상, 테스트가 쉬워짐

= D는 다음 할 일 ②, F는 ③

진행 순서



E(연출 분리)는 이벤트가 늘어 VFX 코드가 반복될 때 끼워 넣음